



Sorting Algorithms

Comparison-Based & Non-Comparison Sorting

Java 8+



Session Structure

1. Why sorting matters
2. Basic sorting algorithms
3. Efficient sorting algorithms
4. Divide & Conquer
5. Non-comparison sorts
6. Java built-in sorting
7. Real-world systems



Why Sorting?

Sorting enables:

- Faster searching
- Data organization
- Efficient merging
- Better visualization
- Algorithm optimization

Complexity Overview

Algorithm	Best	Average	Worst	Space
Bubble	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Heap	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$



Bubble Sort

Idea:

Repeatedly swap adjacent elements.

```
static void bubbleSort(int[] arr) {  
    int n = arr.length;  
    for (int i = 0; i < n - 1; i++) {  
        boolean swapped = false;  
        for (int j = 0; j < n - i - 1; j++) {  
            if (arr[j] > arr[j + 1]) {  
                int temp = arr[j];  
                arr[j] = arr[j + 1];  
                arr[j + 1] = temp;  
                swapped = true;  
            }  
        }  
        if (!swapped) break;  
    }  
}
```

Time: $O(n^2)$



Selection Sort

Idea:

Select minimum each iteration.

```
static void selectionSort(int[] arr) {  
    int n = arr.length;  
  
    for (int i = 0; i < n - 1; i++) {  
        int minIndex = i;  
  
        for (int j = i + 1; j < n; j++)  
            if (arr[j] < arr[minIndex])  
                minIndex = j;  
  
        int temp = arr[minIndex];  
        arr[minIndex] = arr[i];  
        arr[i] = temp;  
    }  
}
```

Time: $O(n^2)$

Insertion Sort

Best for nearly sorted arrays.

```
static void insertionSort(int[] arr) {  
    for (int i = 1; i < arr.length; i++) {  
        int key = arr[i];  
        int j = i - 1;  
  
        while (j >= 0 && arr[j] > key) {  
            arr[j + 1] = arr[j];  
            j--;  
        }  
        arr[j + 1] = key;  
    }  
}
```

Best: $O(n)$

Worst: $O(n^2)$

Merge Sort (Divide & Conquer)

Split → Sort → Merge.

```
static void mergeSort(int[] arr, int left, int right) {  
    if (left >= right) return;  
  
    int mid = left + (right - left) / 2;  
  
    mergeSort(arr, left, mid);  
    mergeSort(arr, mid + 1, right);  
    merge(arr, left, mid, right);  
}
```



```
static void merge(int[] arr, int left, int mid, int right) {  
    int[] temp = new int[right - left + 1];  
  
    int i = left, j = mid + 1, k = 0;  
  
    while (i <= mid && j <= right)  
        temp[k++] = (arr[i] <= arr[j]) ? arr[i++] : arr[j++];  
  
    while (i <= mid) temp[k++] = arr[i++];  
    while (j <= right) temp[k++] = arr[j++];  
  
    System.arraycopy(temp, 0, arr, left, temp.length);  
}
```

Time: $O(n \log n)$

Space: $O(n)$

Quick Sort

Choose pivot → Partition.

```
static void quickSort(int[] arr, int low, int high) {  
    if (low < high) {  
        int pi = partition(arr, low, high);  
        quickSort(arr, low, pi - 1);  
        quickSort(arr, pi + 1, high);  
    }  
}
```

```
static int partition(int[] arr, int low, int high) {  
    int pivot = arr[high];  
    int i = low - 1;  
  
    for (int j = low; j < high; j++) {  
        if (arr[j] < pivot) {  
            i++;  
            int temp = arr[i];  
            arr[i] = arr[j];  
            arr[j] = temp;  
        }  
    }  
  
    int temp = arr[i + 1];  
    arr[i + 1] = arr[high];  
    arr[high] = temp;  
  
    return i + 1;  
}
```

Average: $O(n \log n)$

Worst: $O(n^2)$



Heap Sort

Build heap → Extract max repeatedly.

Time: $O(n \log n)$

Space: $O(1)$

THIRAVUGAL

Counting Sort (Non-Comparison)

Works when range is small.

```
static void countingSort(int[] arr) {  
    int max = java.util.Arrays.stream(arr).max().getAsInt();  
    int[] count = new int[max + 1];  
  
    for (int num : arr)  
        count[num]++;  
  
    int index = 0;  
    for (int i = 0; i < count.length; i++)  
        while (count[i]-- > 0)  
            arr[index++] = i;  
}
```

Time: $O(n + k)$



Java Built-in Sorting

```
java.util.Arrays.sort(arr);
```

For primitives:

Dual-Pivot QuickSort.

For objects:

TimSort (Merge + Insertion hybrid).



Real World Use Case — E-Commerce

Sort products by:

- Price
- Rating
- Popularity

Time efficiency critical.



Real World Use Case — Logs Processing

Sort by:

- Timestamp
- Severity

Optimizes search & filtering.



Stability in Sorting

Stable:

Maintains relative order of equal elements.

Stable:

- Merge Sort
- Insertion Sort

Unstable:

- Quick Sort
- Heap Sort



Common Mistakes

- Ignoring worst case
- Wrong partition logic
- Forgetting base case
- Misunderstanding stability

Interview Discussion Points

- Why QuickSort is preferred?
- Why MergeSort needs extra space?
- Why $O(n \log n)$ is optimal comparison bound?
- When to use Counting Sort?
- Why TimSort in Java?



Lower Bound of Sorting

Comparison-based sorting:

Minimum possible:

$O(n \log n)$

Proof via decision tree.



Performance Thought Exercise

1 million elements:

Bubble:

~ 10^{12} operations.

Merge:

~20 million operations.

Huge scalability gap.

Summary

Sorting is foundational for:

- Searching optimization
- Data organization
- Performance engineering

Master:

- Merge Sort
- Quick Sort
- Heap Sort
- Stability concepts
- $O(n \log n)$ reasoning